



OPERATING SYSTEMS & NETWORKING

Process Scheduling and
Subnetting Operations

KARRNIDH ARORA, NAVNEET DHILLON, KHUSHI MATHUR, JIYA PATHAK, AREZAH SHEIKH

THE OBJECTIVE

PROBLEM

How do classic theoretical algorithms (like SJF, FCFS) perform when applied to the dynamic, real-time workload of a live Linux system, and how do their trade-offs manifest in performance metrics?

How do classic theoretical algorithms (like SJF, FCFS) perform when applied to the dynamic, real-time workload of a live Linux system, and how do their trade-offs manifest in performance metrics?

GOALS

Bridge Theory & Practice: Implement core scheduling algorithms in Python.

Live Integration: Drive simulations using real Linux kernel data (ps, ETIMES).

Automate Validation: Create a tool for subnet calculation and automated reachability verification.

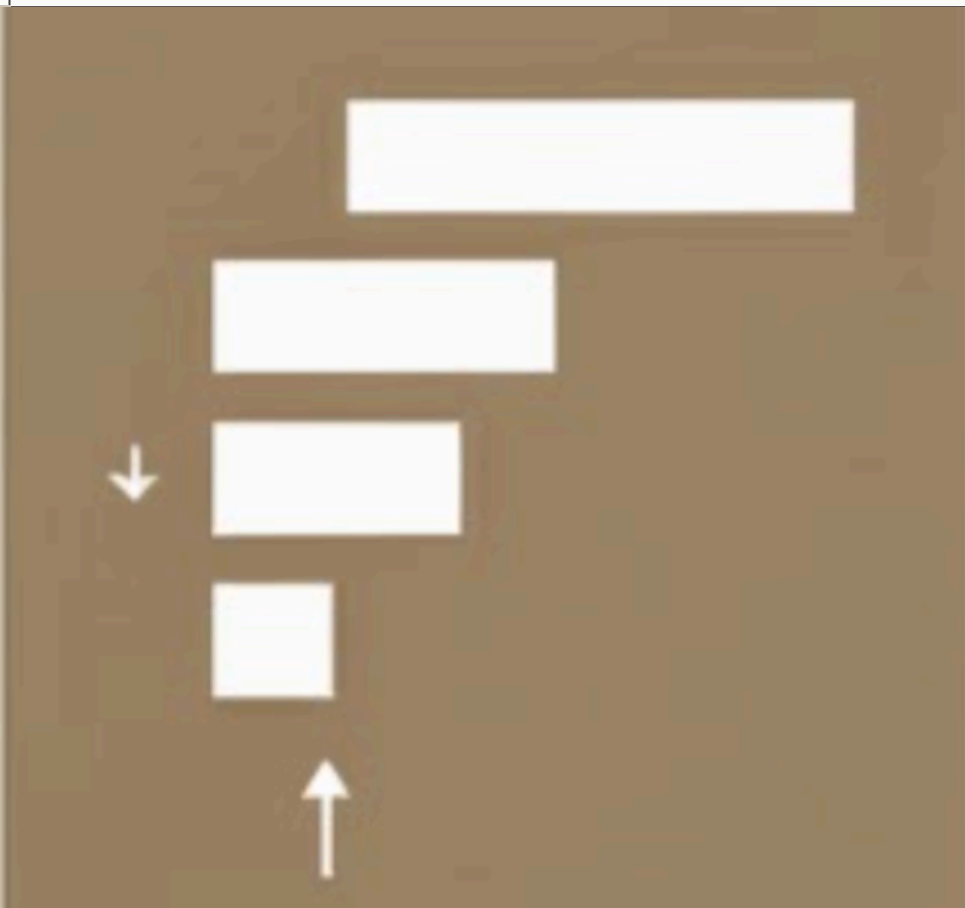
PROCESS SCHEDULING

FIRST COME FIRST SERVE (FCFS)



Simple, non-preemptive. Suffers from the "Convoy Effect" if long processes arrive first.

SHORTEST JOB FIRST (SJF)



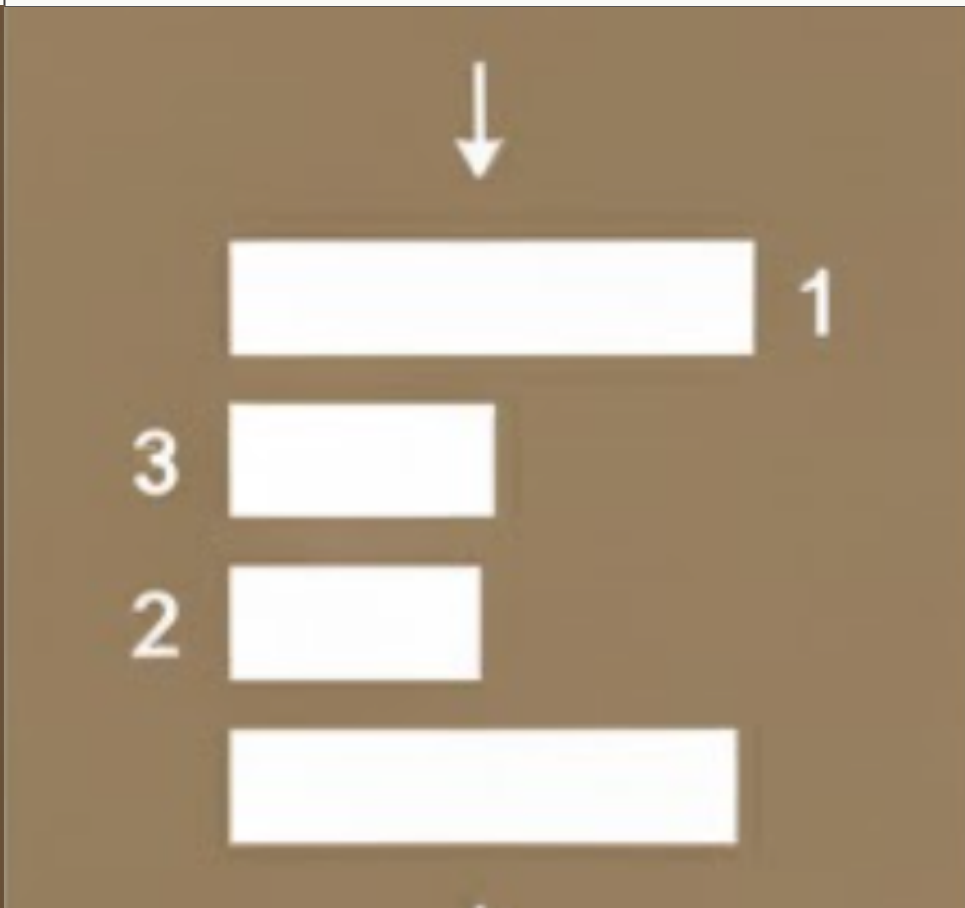
Selects the process with the smallest execution time. Optimizes for minimal waiting time.

ROUND ROBIN



Preemptive. Each process gets a fixed time slice (Quantum=3). Essential for system responsiveness.

PRIOROTY SCHEDULING



Non-preemptive in our simulation. Selects the job based on its OS priority value. Risks starvation for low-priority tasks.

IMPLEMENTATION ARCHITECTURE

Python Simulation Logic

We designed a robust pipeline to process system data:

- The Job Object (PCB): A Class structure holding PID, Name, Burst Time, and Priority.
- The Baseline: All 4 algorithms run on a fresh copy of the exact same process list, guaranteeing fair comparison.\
- Simulation Clock: A central time variable (t) advances as processes execute, calculating start and completion times.



SCHEDULING ON PYTHON

--- FCFS ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	1	procA	5	0	5	0	5	20
1	2	procB	3	5	8	5	8	18
2	3	procC	8	8	16	8	16	22
3	4	procD	6	16	22	16	22	19
4	5	procE	2	22	24	22	24	21

Avg Waiting = 10.20, Avg Turnaround = 15.00

Saved Gantt chart to: GanttChartFCFS.png

--- SJF ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	5	procE	2	0	2	0	2	21
1	2	procB	3	2	5	2	5	18
2	1	procA	5	5	10	5	10	20
3	4	procD	6	10	16	10	16	19
4	3	procC	8	16	24	16	24	22

Avg Waiting = 6.60, Avg Turnaround = 11.40

--- Round Robin ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	1	procA	5	0	16	11	16	20
1	2	procB	3	3	6	3	6	18
2	3	procC	8	6	24	16	24	22
3	4	procD	6	9	22	16	22	19
4	5	procE	2	12	14	12	14	21

Avg Waiting = 11.60, Avg Turnaround = 16.40

Saved Gantt chart to: GanttChartRoundRobin.png

--- Priority ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	2	procB	3	0	3	0	3	18
1	4	procD	6	3	9	3	9	19
2	1	procA	5	9	14	9	14	20
3	5	procE	2	14	16	14	16	21
4	3	procC	8	16	24	16	24	22

Avg Waiting = 8.40, Avg Turnaround = 13.20

Saved Gantt chart to: GanttChartPriority.png

Summary Comparison:

	Algorithm	Avg Waiting	Avg Turnaround
0	FCFS	10.2	15.0
1	SJF	6.6	11.4
2	Round Robin	11.6	16.4
3	Priority	8.4	13.2

- SJF performs best among all algorithms, achieving the lowest average waiting (6.6) and turnaround time (11.4).
- Priority scheduling is the next most efficient, balancing fairness and performance (Avg Waiting 8.4).
- Round Robin shows higher delays due to time-slicing overhead but ensures equal CPU time for all processes.

LINUX INTEGRATION



FETCHING LIVE KERNEL DATA

We use the subprocess module to query the OS kernel directly, avoiding random number generation.

```
cmd = ["ps", "-eo", "pid,comm,etimes,pri", "--  
sort=-pcpu"]
```

MAPPING STRATEGY

ETIMES

(Elapsed Time) → Burst Time

PRI

(Kernel Priority) → Priority

SCHEDULING ON LINUX

Live Linux processes:

	PID	COMMAND	ETIMES	PRI	NI	PCPU
0	1597	python3	0	19	0	101.0
1	1	systemd	505	19	0	0.4
2	1590	kworker/1:19-xf	27	19	0	0.1
3	13	kworker/u8:1-ev	505	19	0	0.1
4	1340	sshd-session	335	19	0	0.0

--- FCFS ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	1597	python3	1	0	1	0	1	19
1	1	systemd	20	1	21	1	21	19
2	1590	kworker/1:19-xf	20	21	41	21	41	19
3	13	kworker/u8:1-ev	20	41	61	41	61	19
4	1340	sshd-session	20	61	81	61	81	19

Avg Waiting = 24.80, Avg Turnaround = 41.00

Saved Gantt chart to: GanttChartFCFS.png

--- SJF ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	1597	python3	1	0	1	0	1	19
1	1	systemd	20	1	21	1	21	19
2	1590	kworker/1:19-xf	20	21	41	21	41	19
3	13	kworker/u8:1-ev	20	41	61	41	61	19
4	1340	sshd-session	20	61	81	61	81	19

Avg Waiting = 24.80, Avg Turnaround = 41.00

Saved Gantt chart to: GanttChartSJF.png

--- Round Robin ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	1597	python3	1	0	1	0	1	19
1	1	systemd	20	1	75	55	75	19
2	1590	kworker/1:19-xf	20	4	77	57	77	19
3	13	kworker/u8:1-ev	20	7	79	59	79	19
4	1340	sshd-session	20	10	81	61	81	19

Avg Waiting = 46.40, Avg Turnaround = 62.60

Saved Gantt chart to: GanttChartRoundRobin.png

--- Priority ---

	PID	Name	Burst	Start	Completion	Waiting	Turnaround	Priority
0	1597	python3	1	0	1	0	1	19
1	1	systemd	20	1	21	1	21	19
2	1590	kworker/1:19-xf	20	21	41	21	41	19
3	13	kworker/u8:1-ev	20	41	61	41	61	19
4	1340	sshd-session	20	61	81	61	81	19

Avg Waiting = 24.80, Avg Turnaround = 41.00

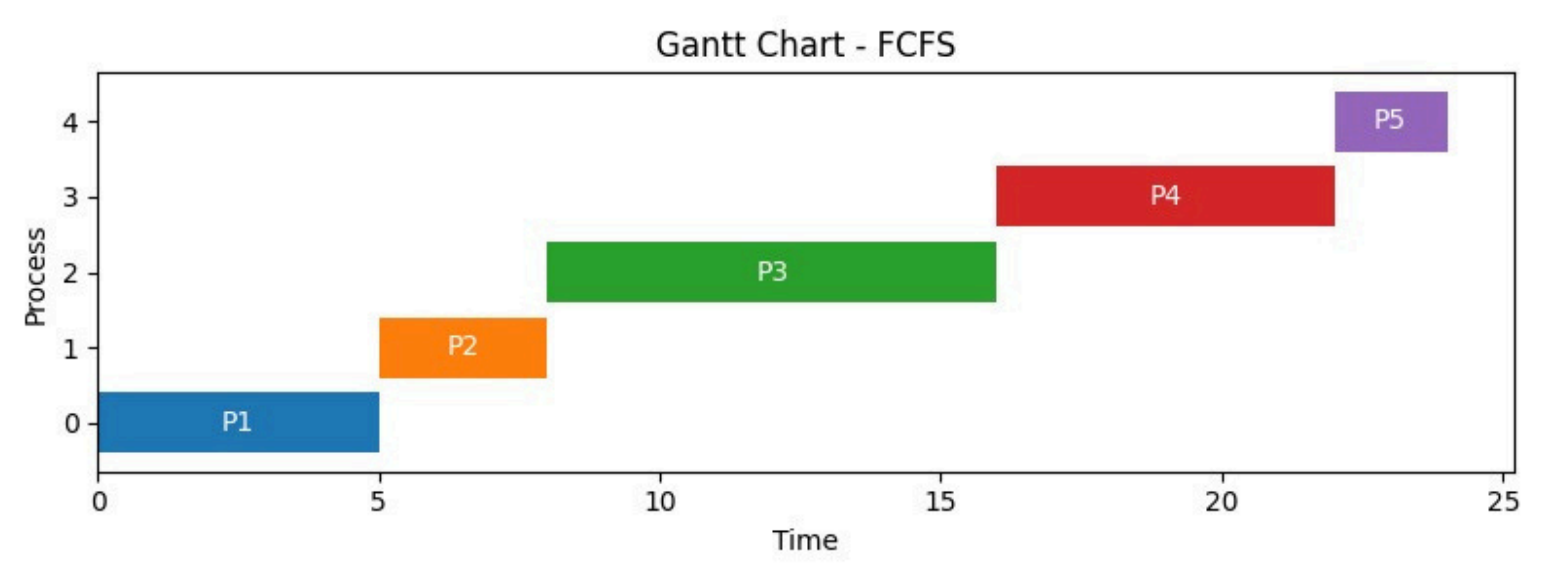
Saved Gantt chart to: GanttChartPriority.png

Summary Comparison:

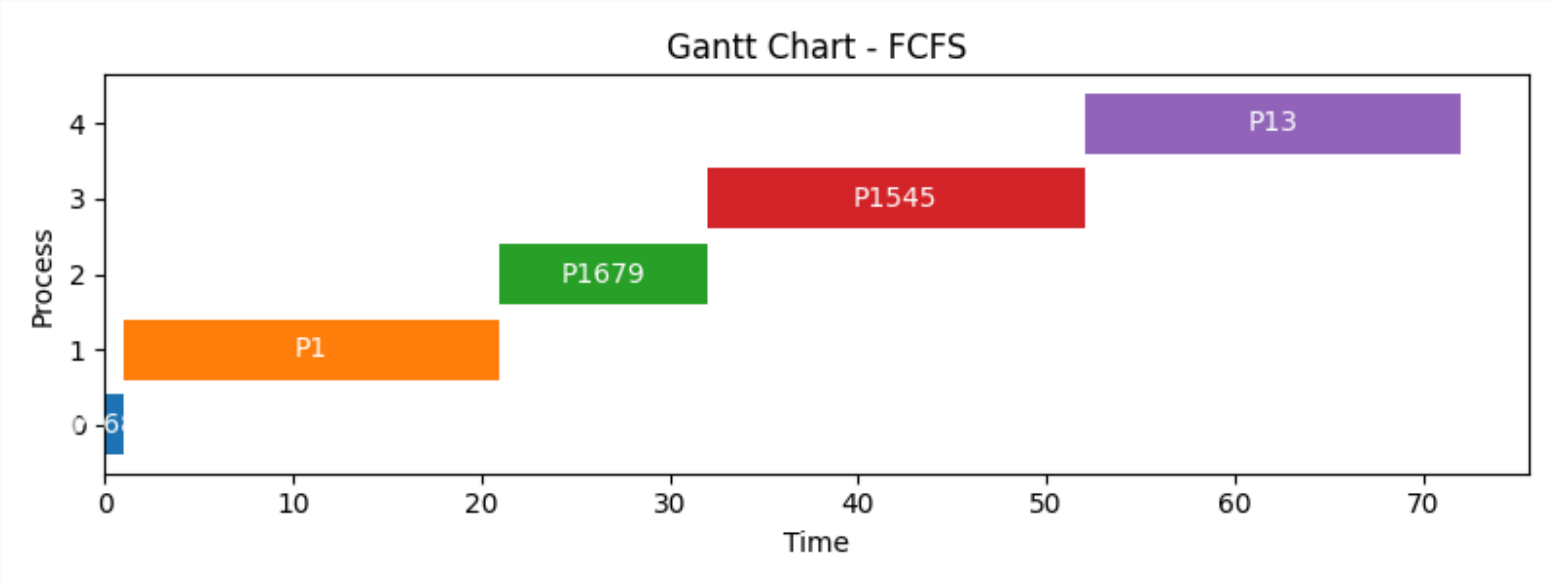
	Algorithm	Avg Waiting	Avg Turnaround
0	FCFS	24.8	41.0
1	SJF	24.8	41.0
2	Round Robin	46.4	62.6
3	Priority	24.8	41.0

[ec2-user@ip-172-31-77-13 OS_project]\$ python3 subnetting.py

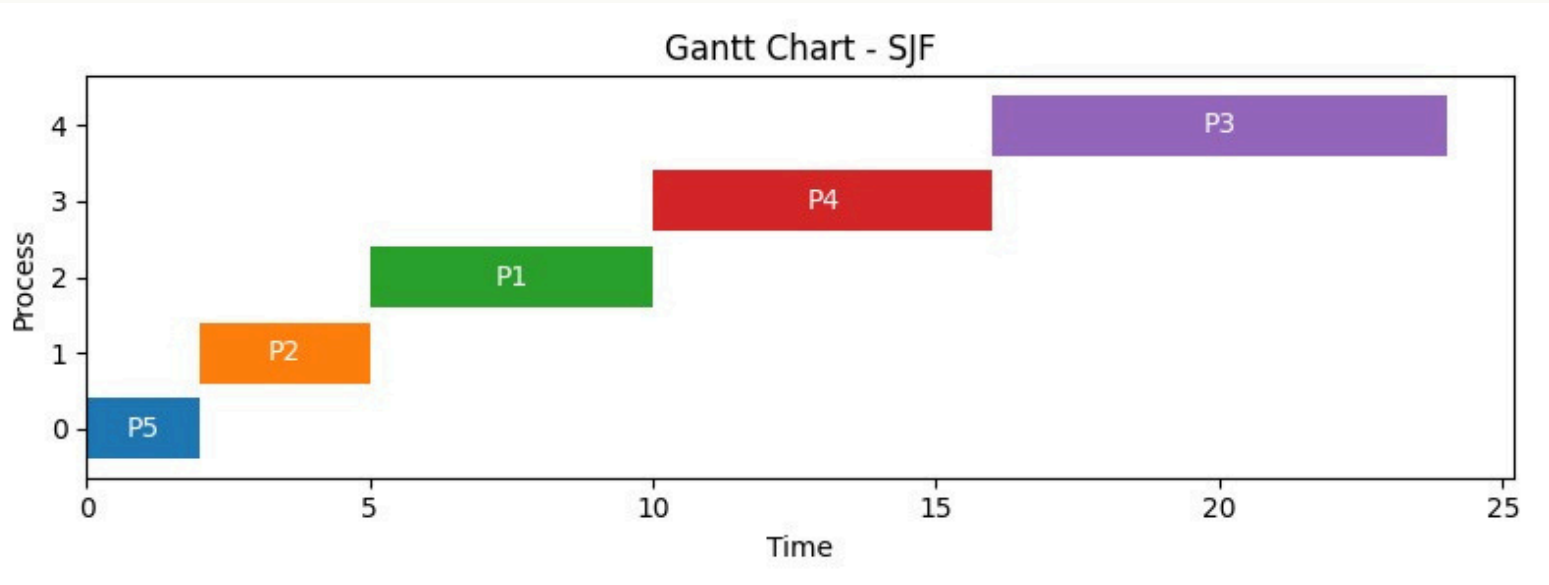
RESULTS AND ANALYSIS



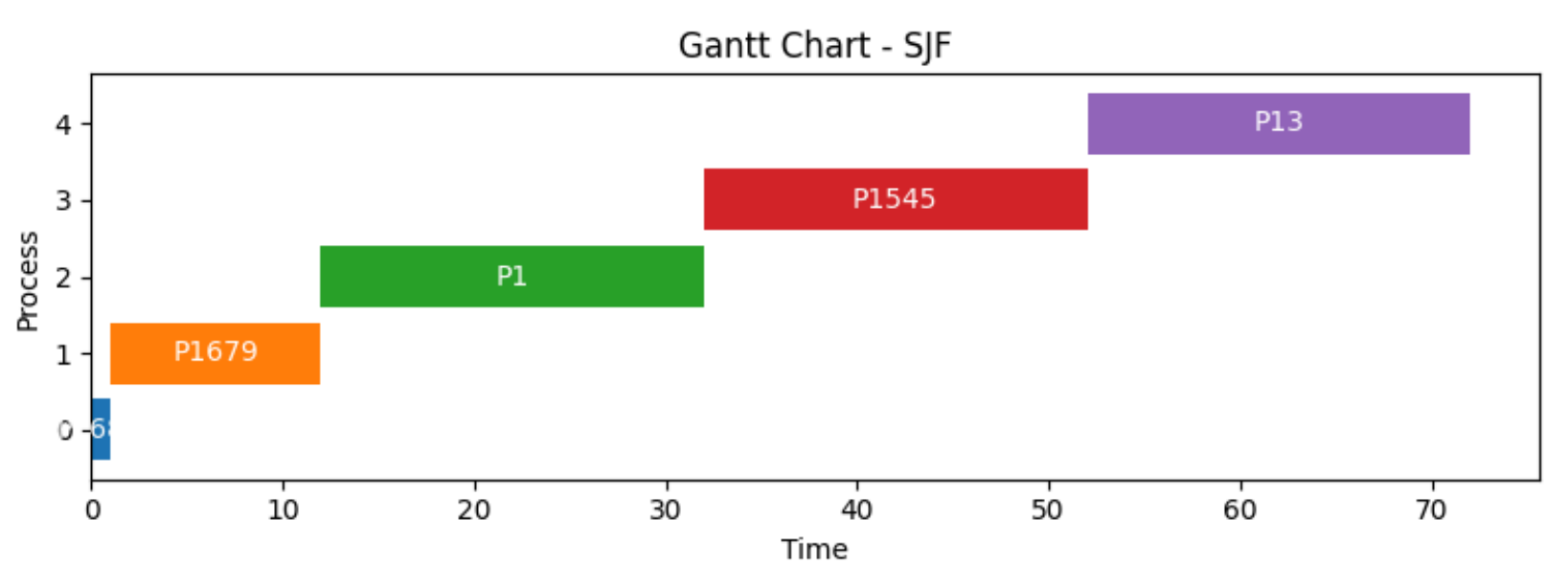
FCFS GANTT CHART ON PYTHON



FCFS GANTT CHART ON LINUX

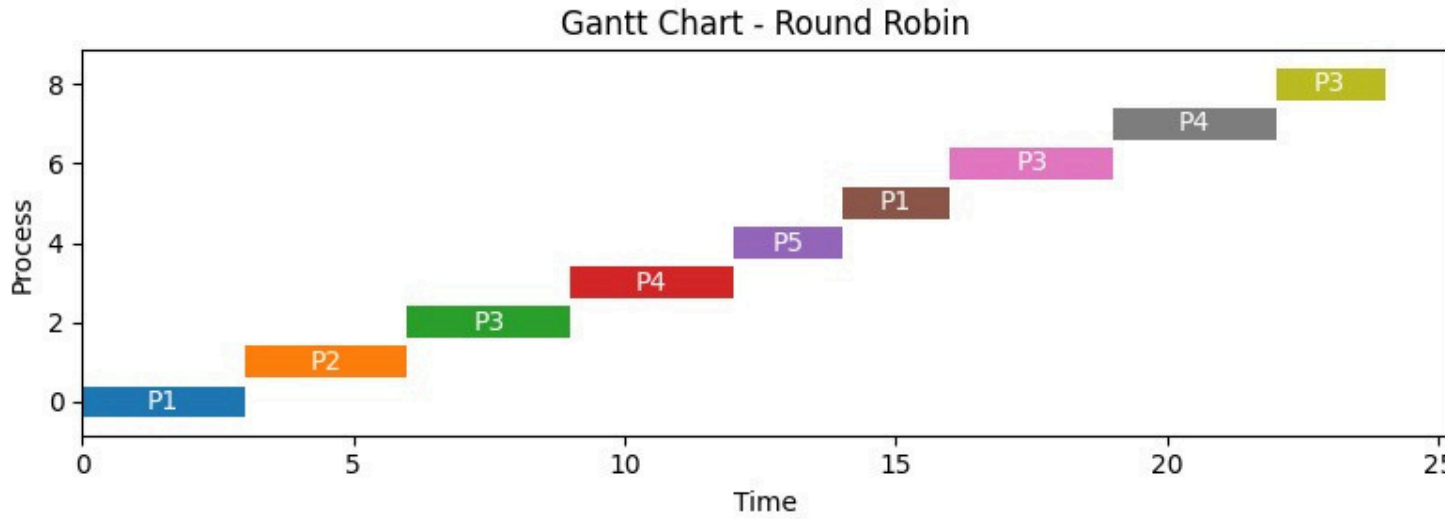


SJF GANTT CHART ON PYTHON

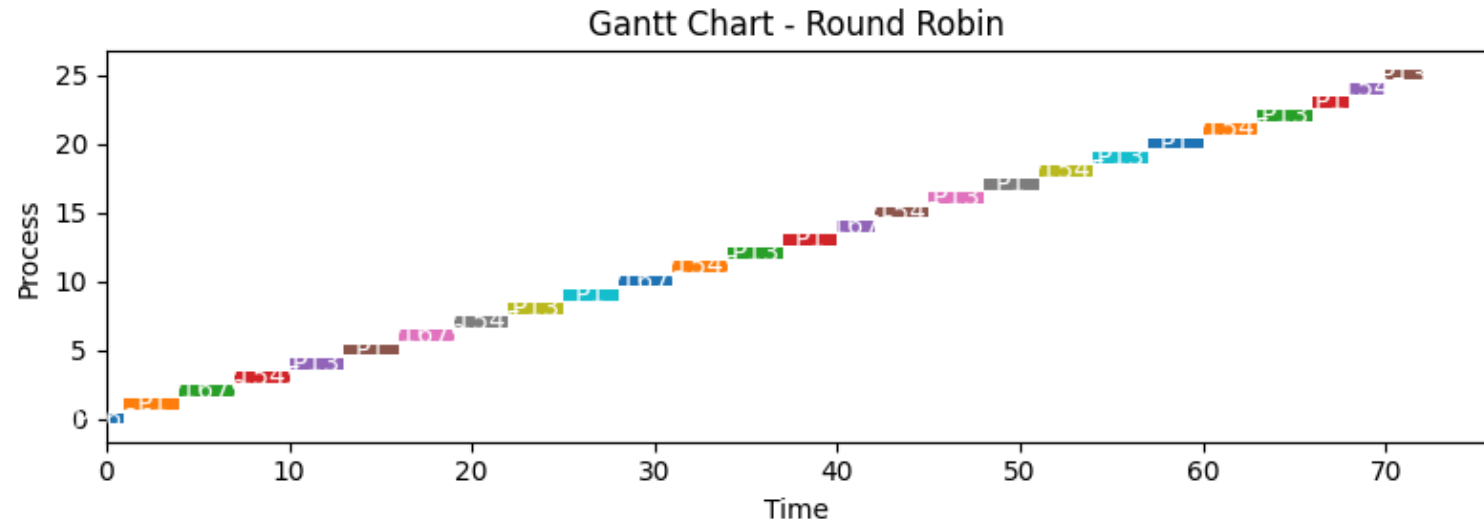


SJF GANTT CHART ON LINUX

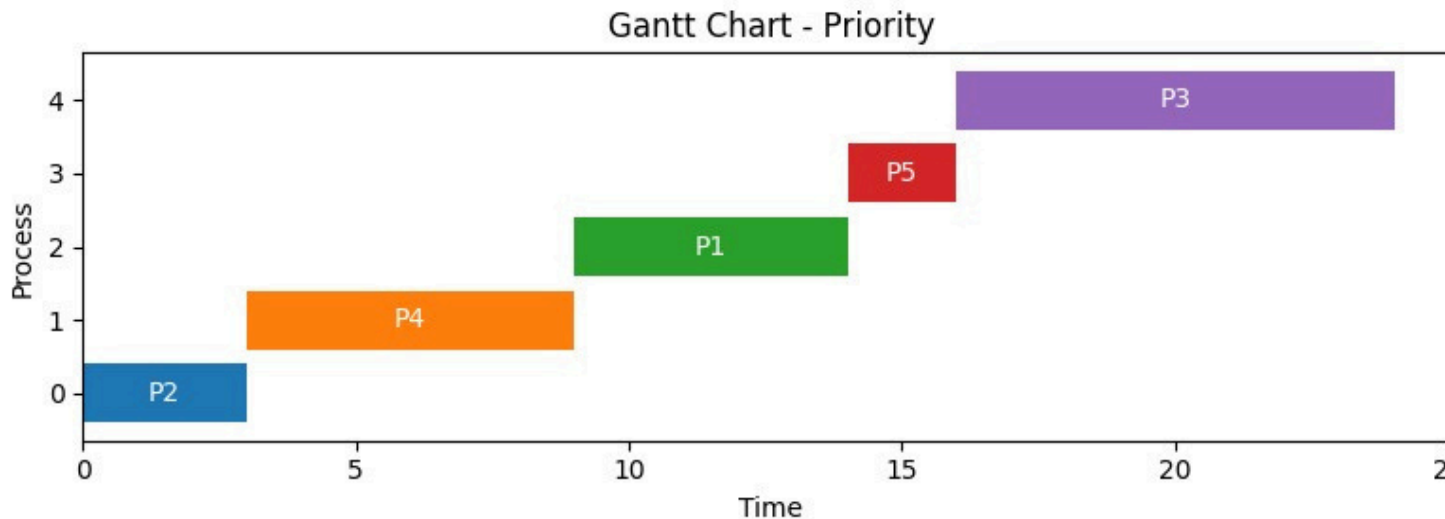
RESULTS AND ANALYSIS



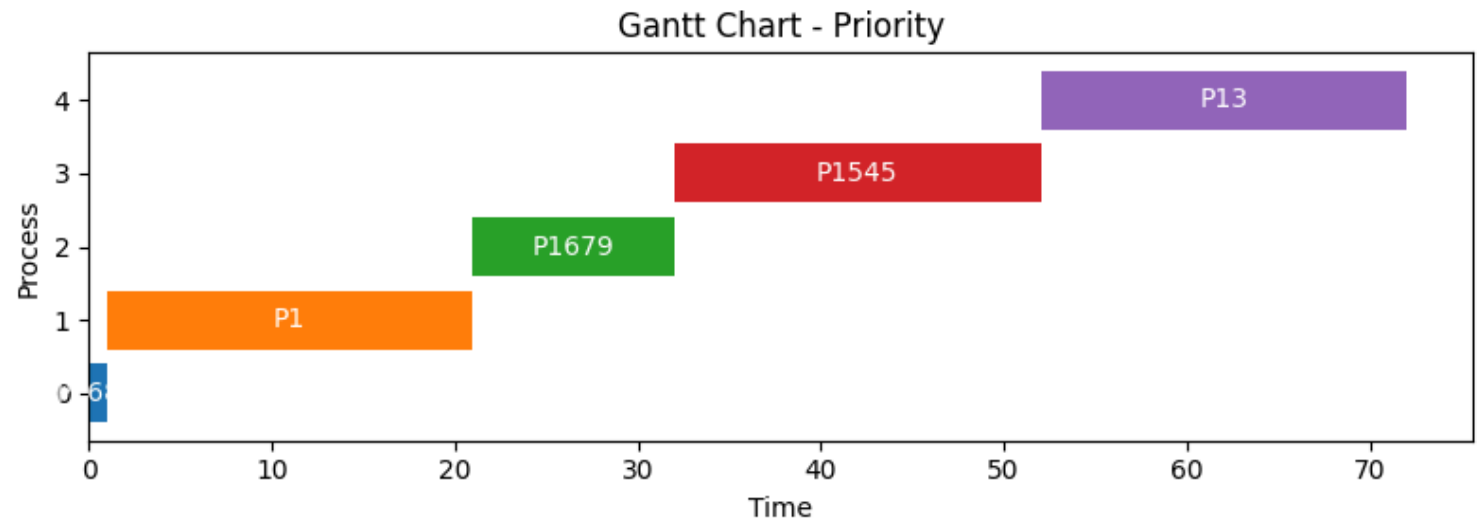
ROUND ROBIN GANT CHART ON PYTHON



ROUND ROBIN GANT CHART ON LINUX



PRIORITY GANTT CHART ON PYTHON



PRIORITY GANTT CHART ON LINUX

SUBNETTING OPERATIONS

Subnetting means dividing an IP network into more manageable sub-networks, or subnets.

THE ALGORITHM CALCULATES



It automatically does these calculations using Python's IP address module and then checks the results against real system configurations using network commands like ip addr show, ipconfig, and ping.

```
=== SUBNETTING TOOL ===

Enter IP address with CIDR (e.g., 192.168.1.0/24):  192.168.1.0/24
Enter required hosts per subnet:  6

INPUT
IP Network: 192.168.1.0/24
Required hosts per subnet: 6

OUTPUT
Original CIDR:          24
New CIDR:               29
Subnet Mask:            255.255.255.248
Number of Subnets:     32
Usable Hosts per Subnet: 6

SUBNET DETAILS

Subnet 1:
Network Address: 192.168.1.0
Broadcast Address: 192.168.1.7
Host Range: 192.168.1.1 - 192.168.1.6
Subnet Mask: 255.255.255.248
Total Hosts: 6

Subnet 2:
Network Address: 192.168.1.8
Broadcast Address: 192.168.1.15
Host Range: 192.168.1.9 - 192.168.1.14
Subnet Mask: 255.255.255.248
Total Hosts: 6
```

TEAM CONTRIBUTION

KARRNIDH ARORA

Scheduling Code, Github, PPT

NAVNEET DHILLON

Scheduling Code, Github, PPT

JIYA PATHAK

Documentation, ReadMe, PPT

KHUSHI MATHUR

Subnetting Code, ReadMe, PPT

AREZAH SHEIKH

PPT

THANK YOU